Section 3.1: The XOR Problem

Why Linear Models Are Not Enough

AI Class — Lecture Notes

---

1 The XOR Dataset

The **exclusive OR** (XOR) function takes two binary inputs and outputs 1 if and only if *exactly one* of them is 1:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Notice that this is the same four input points used in the linear regression example of Section 1.1, but the labels have changed: instead of $y = x_1 + x_2$ (a perfectly linear relationship), the output is now $y = x_1 \oplus x_2$ (XOR), which is *non-linear*.

The question we will answer in this section is: **Can a linear model learn XOR?** And if not — what can?

2 Linear Regression Fails on XOR

Applying the normal equations $\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ to the XOR dataset:

$$\mathbf{X} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}.$$

Computing:

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 2 & 1 \\ 2 & 1 & 2 \end{bmatrix}, \qquad \mathbf{X}^\top \mathbf{y} = \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}.$$

Solving $(\mathbf{X}^\top \mathbf{X})\, \boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y}$ gives:

$$\boldsymbol{\theta} = \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0 \\ 0 \end{bmatrix}$$

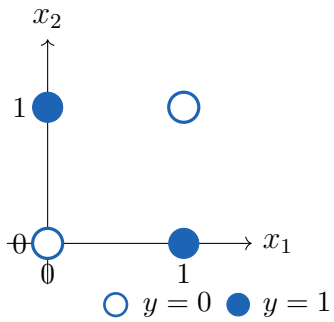
The model predicts $\hat{y} = 0.5$ for *every* input, regardless of x_1 and x_2. The resulting MSE is:

$$\text{MSE} = \frac{(0.5)^2 + (-0.5)^2 + (-0.5)^2 + (0.5)^2}{4} = 0.25.$$

This is the best any linear regression model can do on XOR. No choice of w_1, w_2, b can reduce the MSE below 0.25 — not because the model is poorly tuned, but because the relationship is fundamentally non-linear.

3 Logistic Regression Also Fails

Logistic regression tries to separate the two classes with a **linear decision boundary**: the line $w_1 x_1 + w_2 x_2 + b = 0$.



The two classes are arranged in a **checkerboard pattern**: no single straight line can put all the filled dots on one side and all the open dots on the other.

XOR is **not linearly separable**. Any linear classifier — whether linear regression or logistic regression — will produce a decision boundary that misclassifies at least two of the four points. The best logistic regression can do is predict $\hat{p} = 0.5$ everywhere (cross-entropy loss $= \ln 2 \approx 0.693$ per sample).

4 The Key Insight: Feature Transformation

What if, instead of feeding (x_1, x_2) directly to a linear classifier, we first **transform** the inputs into a new space where XOR *is* linearly separable?

Define two new features:

$$h_1 = \text{step}(x_1 + x_2 - 0.5) \quad \text{(detects: at least one input is 1)}$$
$$h_2 = \text{step}(x_1 + x_2 - 1.5) \quad \text{(detects: both inputs are 1)}$$

where $\text{step}(z) = 1$ if $z \geq 0$, else 0. Computing these for all four inputs:

x_1	x_2	h_1	h_2	y
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

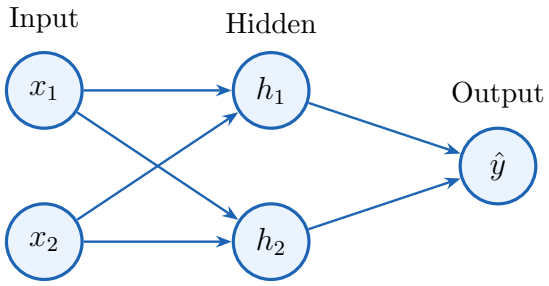
In the new feature space (h_1, h_2) we need to output 1 when $(h_1, h_2) = (1, 0)$ and 0 otherwise. This *is* linearly separable:

$$y = h_1 - h_2.$$

Insight. The hidden features h_1 and h_2 compute an OR gate and an AND gate, respectively. XOR is OR minus AND. The non-linear step function in each neuron is what makes the new representation useful — a purely linear transformation of (x_1, x_2) cannot produce a linearly separable space for XOR.

5 A Feedforward Neural Network

The transformation above has a natural network structure:



5.1 Forward Pass

Let g denote the hidden activation function (e.g. step, sigmoid, or ReLU). The computation proceeds in two steps:

Hidden layer — apply a linear transformation, then a non-linearity:

$$\mathbf{h} = g\big(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\big)\,, \tag{1}$$

where $\mathbf{W}^{(1)} \in \mathbb{R}^{2\times 2}$ and $\mathbf{b}^{(1)} \in \mathbb{R}^2$ are the weights and biases of the hidden layer.

Output layer — a linear combination of the hidden features:

$$\hat{y} = \mathbf{W}^{(2)}\mathbf{h} + b^{(2)}. \tag{2}$$

5.2 Weights That Solve XOR

The construction in Section 4 corresponds to the following network parameters:

$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} -0.5 \\ -1.5 \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} 1 & -1 \end{bmatrix}, \quad b^{(2)} = 0.$$

Verify for the input $(x_1, x_2) = (1, 1)$:

$$\mathbf{h} = \text{step}\left(\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} -0.5 \\ -1.5 \end{bmatrix} \right) = \text{step}\left(\begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix} \right) = \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

$$\hat{y} = \begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 0. \quad \checkmark$$

6 From Hand-Crafted to Learned Features

The weights above were designed by hand. In practice, with high-dimensional data and complex patterns, we cannot design features manually.

The power of a neural network is that the **weights are learned from data** using gradient descent on a suitable loss function. The network discovers which hidden features are useful, automatically.

Why depth matters. A network with *no* hidden layer is just a linear model. Adding even *one* hidden layer with a non-linear activation turns the network into a **universal function approximator**: given enough neurons, it can approximate any continuous function to arbitrary precision (Universal Approximation Theorem).

7 Summary

Problem	XOR is not linearly separable
Linear models	Both linear and logistic regression predict a constant
Fix	Add a hidden layer with a non-linear activation
Key idea	Hidden neurons learn a new feature space where the task is linearly separable
Forward pass	$\mathbf{h} = g(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \ \hat{y} = \mathbf{W}^{(2)}\mathbf{h} + b^{(2)}$

Up next (Section 3.2): How do we *train* a feedforward network? We need to compute gradients of the loss with respect to every weight in the network — this is the **backpropagation** algorithm.